Exercise sheet 2[1]

Coding exercises

Q2.1 Write a function that takes the arguments a, b and c, and displays the roots of the quadratic equation

$$ax^2 + bx + c = 0. \tag{1}$$

If the roots are complex, display the results in the form $A \pm B\, i$. Look in the documentation of the `<cmath>` library (`http://en.cppreference.com/w/cpp/header/cmath`) for a square-root function.

Q2.2 Write functions to evaluate the following expressions:

$$f(\gamma) = 2\tan^{-1}\left[\left(\frac{1+\gamma}{1-\gamma}\right)^{1/2}\sin\gamma\right], \qquad g(\gamma) = \frac{1}{\sqrt{2\pi}}\log\left(\gamma^2+1\right)e^{-\gamma^2}.$$

Evaluate and display the values of

$$f(g(x)) \qquad \text{and} \qquad g(f(x+a)+g(b)),$$

where $x = 0.15$, $a = 0.235$ and $b = 1.763$. Recall that many of the standard mathematical functions are implemented in the `<cmath>` header: see `http://en.cppreference.com/w/cpp/header/cmath`.

Q2.3 Write a program that displays two columns of numbers, x and x^2, where $x = 0, 1/3, 2/3, \ldots, 2$. The columns must be aligned, e.g.

```
0         0
0.333333  0.111111
0.666667  0.444444
1         1
1.33333   1.77778
1.66667   2.77778
2         4
```

Q2.4 Write a program that takes a positive integer n specified by the user, and writes a text file containing the numbers 1 to n, one on each line of the file.

Q2.5 (a) Write functions to calculate the Fibonacci numbers using

- 32-bit integer (`int`),
- 64-bit integer (`long long`),
- 32-bit floating point (`float`), and
- 64-bit floating point (`double`)

[1] Version 28/09/2026.
Feedback, comments, corrections, applause, etc. to: `M.Heil@maths.manchester.ac.uk`

28/09/2026, 07:59:00

data types[2].

(b) In each case, describe what happens to the results for large enough n (you may need to look up to about $n = 100$). To display all the digits of the floating-point variables, use `std::cout << std::fixed;`, Given that the Fibonacci numbers are actually (mathematically) integers you can suppress the digits after the decimal point using the statement `std::cout.precision(0);`.

Q2.6 Consider the following sequence of natural numbers, known as the Collatz numbers:

$$c_{n+1} = \begin{cases} 3c_n + 1 & \text{if } c_n \text{ is odd,} \\ c_n/2 & \text{if } c_n \text{ is even,} \end{cases} \tag{2}$$

where we start the sequence from any integer $c_0 > 0$.

(a) Write a program that displays successive terms of this sequence, starting at a given value of c_0. You may find it useful to use the remainder operator, `%`: the expression `a%b` evaluates to the remainder of `a` when divided by `b` (a number between `0` and `b-1` – see `https://en.cppreference.com/w/cpp/language/operator_arithmetic`).

(b) Modify your program to allow the user to input a value of c_0 through `std::cin`.

(c) You can convince yourself that if the sequence ever reaches 1, subsequent terms will be 4, 2, 1, 4, 2, ..., and the sequence becomes periodic. Modify your program to stop displaying the numbers in the sequence when $c_n = 1$.

(d) Write a function that calculates and returns the smallest n such that $c_n = 1$. We call this the *Collatz sequence length* of c_0. Use this function to write a program that, for each initial number c_0 from 1 to 30, outputs the Collatz sequence length of c_0.

The Collatz conjecture is one of the most famous unsolved problems in mathematics, and it states that for any initial number $c_0 > 0$, the sequence eventually reaches 1. While the conjecture is still open, it has been checked numerically for $c_0 < 2^{68} \approx 2.95 \times 10^{20}$, so you may assume (!) that the sequence will eventually reach 1!

[2] `long long` is a C++11 feature, so may not be available on all compilers. The sizes of `int` and `long long` are not guaranteed to be 32 and 64 bits respectively, but this is often the case.

28/09/2026, 07:59:00

Analysis exercise

Analysis Exercise 2: Newton's method This week's analysis exercise concerns Newton's method for finding the roots x of a nonlinear equation, $f(x)$, i.e. the values of x for which $f(x) = 0$. Starting from some initial guess for the location of the root, x_0, the Newton method produces the iterates

$$x_{n+1} = x_n - \frac{f(x_n)}{f'(x_n)},$$

for $n = 0, 1, 2, \ldots$. If the initial guess is close enough to a root of f, then the sequence x_n converges to it.
Below we provide a simple implementation of this algorithm for the function

$$f(x) = 9x^4 - 42x^3 - 1040x^2 + 5082x - 5929.$$

We have split the code into two files:

- The Newton solver itself is defined in a so-called header file, `newton_solver.h`, which only includes the function that performs the Newton iteration.
- The header file is then included (using the `#include` statement that you're familiar with from the inclusion of C++ headers such as `#include<iostream>`) into the "driver code" `newton_solver.cpp`.

When compiling the driver code, the compiler literally *includes* the code in `newton_solver.h` into `newton_solver.cpp` and then compiles the resulting file to an executable, so it behaves as if the code in the header file had been copied and pasted into the driver code.
The advantage of this approach is that the header file contains generic functionality that may be useful in other applications too. For this to work, we must give the Newton solver access to the functions $f(x)$ and $f'(x)$ whose C++ implementations are functions that take a `double` as an argument and then return a `double`. The first two arguments of the function `newton_solver(...)` in `newton_solver.h` show how this is done:

The header file `newton_solver.h`:

```
// Header file containing the code for the Newton solver

// C++ includes
#include <iostream>
#include <cmath>

// Find the root of the function f(x), starting from an
// initial guess x_init. At most max_iter iterations are
// performed, and the iteration is deemed to have converged
// if |f(x)| < tol. The first two arguments of the function
// are "function pointers" which specify the function f(x)
```


28/09/2026, 07:59:00

```
// (a C++ function that takes a double as an argument and
// returns a double) and its derivative, f'(x) (which has the
// same C++ "signature").
double newton_solver(double fct(double),
                     double deriv(double),
                     double x_init,
                     unsigned max_iter,
                     double tol)
{
 // Initialise the iterate
 double x=x_init;

 // Keep iterating until we converge (in which case we
 // return early) or until we've used up all available
 // iterations
 for (unsigned iter=0; iter<max_iter; iter++)
  {
   // Perform Newton iteration step; note how we're using the
   // functions that were passed in through the function's
   // arguments.
   x -= fct(x)/deriv(x);

   // If |f(x)| < tol, we are close enough to the root,
   // so return x
   if (std::abs(fct(x)) < tol) return x;
  }

 // If we have not returned already, the iteration has not
 // converged and we bail out and abort the program.
 abort();
}
```

The driver code `newton_solver.cpp`:

```
// c++ includes
#include <iostream>

// Include code for Newton solver
#include "newton_solver.h"

// The function whose zero we're trying to find: f(x)
double function(double x)
{
 return 9.0*x*x*x*x - 42.0*x*x*x - 1040.0*x*x + 5082.0*x - 5929;
}

// The derivative f'(x) of the function whose zero we're
// trying to find.
```


28/09/2026, 07:59:00

```
double derivative(double x)
{
 return 4.0*9.0*x*x*x - 3.0*42.0*x*x - 2.0*1040.0*x + 5082.0;
}


// Driver code
int main()
{
 // Provide initial guess
 double x_init=0.0;
 std::cout
  << "Please enter an initial guess for the Newton method: ";
 std::cin >> x_init;

 // Call the Newton solver with 100 iterations and a
 // tolerance of 10^-6. Note how the two (global) functions,
 // function(...) and derivative(...) are passed to the solver.
 unsigned max_iter=100;
 double tol=1.0e-6;
 double x = newton_solver(function, derivative,
                          x_init, max_iter, tol);

 // Tell us what we got
 std::cout << "We converged to a solution x = " << x
           << " for which |F(x)| = " << std::abs(function(x))
           << std::endl;

 // Done
 return 0;
}
```

Tasks:

1. Download the files `newton_solver.cpp` and `newton_solver.h` from Canvas. Move both files into the same directory on your computer. Compile the driver code (as discussed above, the code in the header file will simply be included, but it has to be in the same directory). Run the code to explore its behaviour. Specifically, you are given that the polynomial has three distinct roots: $x = \pm 11$ and the repeated root $x = 7/3$. For each of these roots, find (by trial and error) an initial guess x_0 (that obviously shouldn't be equal to the known root!) such that the algorithm converges to this root within a tolerance of 10^{-6}.

2. Modify the `newton_solver` function so that it prints the iteration counter, n, the iterate, x_n, and the corresponding function value $f(x_n)$ (so three numbers in one line, with one line for each iteration) to a file whose name you should be able to specify as an argument to the function. The "prototype" for the function should therefore change to

28/09/2026, 07:59:00

```
double newton_solver(double fct(double),
                     double deriv(double),
                     double x_init,
                     unsigned max_iter,
                     double tol,
                     std::string convergence_history_filename)
```

Hint: When opening an output stream, you have to specify the filename as a C-style string (an array of `char`s for those who know C):. The literal filenames we've provided in the examples considered so far "work", e.g.,

```
std::ofstream outfile("newton_convergence_history.dat");
```

However, specifying a `std::string` won't work (try it!). You'll have to convert the string to a C-style string, which can be done using the (member) function `c_str()` [a]. So, given the string `convergence_history_filename` that is passed into the function, you'll have to open the file like this:

```
std::ofstream outfile(convergence_history_filename.c_str());
```

See, e.g., `https://cplusplus.com/reference/fstream/ofstream/open/` for details.

3. For an initial guess that converges to 7/3, examine the rate of convergence of the iterates to the repeated root i.e. find a formula that describes the qualitative behaviour of $|x_n - 7/3|$ as x_n tends to 7/3. (**Hint:** Plot $|x_n - 7/3|$ vs. n in a semi-log plot, using a logarithmic axis for $|x_n - 7/3|$).

   Compare this with the case for an initial guess that converges to ± 11. Find theoretical results in the literature or online to explain the difference you observe.

4. Change the function to $f(x) = x^2 + 2x + 5$. What happens and why? To handle such cases more gracefully, modify `newton_solver.h` so that before aborting a suitable error message (including the word "ERROR") is written to `std::cout` to explain to the user what happened[b]

Copy the modified header file `newton_solver.h` to a file called `analysis_exercise2.h`. Upload this file, and the pdf file containing your report, `analysis_exercise2.pdf`, to the course's GRADESCOPE page, accessible via Canvas.
It is not necessary to upload the driver code. We will test your header file with our own driver code! Note that for this to work, your header file must not contain the `main()` function!
The pdf file of your report must have **no more than 7 pages**, incl. the title page which must show your student ID. The report should address questions 1, 3 and 4. Our GRADESCOPE-based tests will assess questions 2 and 4.
You can upload your files as many times as you want. The grade for the exercise will be based on the final version received before the submission deadline:

28/09/2026, 07:59:00

14:00 on Tuesday of Week 4.

[a] This is the first time you see an object in action. The `std::string` isn't just a primitive data type (like a `double`, say), but a class that provides "member functions" to do things with it. We'll discuss this in much more detail later in the course.

[b] **STYLE:** Strictly speaking, we should write errors to an alternative stream, `std::cerr`, which you're welcome to do. By default both streams write "to the screen" which is where we currently want the error messages to appear. Later in the course we will discuss how to handle errors more gracefully than by simply aborting the program execution.

28/09/2026, 07:59:00